

CS103-Computer Programming 2
Programming Project
2nd Semester, 2023-2024

PROJECT DOCUMENTATION

Project: Terminal - Based Bus Ticketing System

Members: Melbert Bonina
Maria Francia Abdula
Rhoda Rodriguez
Angelica Bilaos

I. Introduction

Our project aims to develop a straightforward Bus Transport Ticketing System using the C programming language. The system is designed to make ticket booking for bus transportation services easy and convenient for passengers, while also providing operators with the necessary tools to manage routes, fares, and passenger data effectively.

Our system is tailored for stationary deployment, serving as an efficient ticketing solution for bus booths, ensuring consistency in the starting point for every journey within a specified location. The focus is on catering to a single bus company rather than encompassing the entire bus complex within a terminal. This means that our system is designed to meet the needs of a specific bus company, providing tools and functionalities that facilitate ticket booking, route management, fare setting, and passenger data handling. After printing the intended ticket, passengers proceed to the ticket cashier to pay for their tickets.

A. Purpose

The purpose of our Bus Transport Ticketing System is to:

1. Optimize the ticket booking experience for passengers, facilitating route selection, ticket purchase, and ticket retrieval in a seamless manner.
2. streamline route management, fare configuration, and passenger data management, thereby promoting operational efficiency and elevating service quality, for operators.

B. Benefits and Beneficiaries

The primary users of our program are the company's operators and customers. Operators benefit from features that enable efficient route management, fare setting, and passenger data handling, wherein they can streamline administrative tasks, make data-driven decisions, and enhance operational efficiency.

On the other hand, customers benefit from an interface for ticket booking and management. They can easily view available routes, book tickets, and print tickets for their journeys. This simplifies the booking process and enhances the overall experience for passengers.

C. Features/Functions

1. Passenger Menu:

- a. **Ticket Booking:** Passengers can easily find and book bus tickets for their desired routes. They can view routes based on destination locations and availability, select and specify the number of tickets needed, including how many are discounted.
- b. **Route Information:** Passengers have a view of the available locations and its current status.
- c. **Booking Management:** Passengers can manage their bookings efficiently. They can view their current bookings, and after checking their booking, they can either confirm or cancel their booking.
- d. **Ticket Printing:** After confirming their booking, passengers can conveniently print their tickets. The system generates printable tickets with essential details such as the passenger's name, destination and payment details that is stored in a printable file.

2. Operator Menu:

- a. **Route Management:** Operators have full control over route management. They can add new routes to the system, specifying details such as date, route's origin, destination, and distance. Operators can also edit existing routes to update route information or remove routes that are no longer operational.
- b. **Fare Setting:** Operators can set and adjust fares for different bus type categories. They define pricing structures based on factors such as distance traveled, and bus type rate (e.g., ordinary, air-conditioned, deluxe).
- c. **Passenger Data Handling:** The system allows operators to manage passenger data. They can store passenger information, including name, route taken, and their payment. Operators can view passenger information for each ticket, and generate reports for analysis on their command.

II. Algorithm

Below is a pseudocode of the program. This pseudocode focuses on the main functionalities of the program, and the prompt for each menu on this system is simplified or disregarded.

ASSUME that all variables used are declared :

```
SET operatorFile = Open operators.txt for reading
SET numOfOperator = readNumberOfOperators(operatorFile)
CALL readOperatorFile(operatorFile, operatorList, numOfOperator)
CLOSE operatorFile
```

```
SET routeFile = OPEN route.txt for reading
SET numOfRoutes = readNumberOfRoutes(routeFile)
CALL readRouteFile(routeFile, routeList, numOfRoutes)
CLOSE routeFile
```

```
SET pricingFile = OPEN price.txt for reading
SET numOfPrice = readNumberOfPrices(pricingFile)
CALL readPriceFile(pricingFile, priceList, numOfPrice)
CLOSE pricingFile
```

```
SET ticketFile = OPEN tickets.txt for writing
IF ticketFile is NULL THEN
    DISPLAY "Error opening file"
    RETURN 1
END IF
```

```
SET bookingInputFile = OPEN passenger.in for both reading and writing
SET numOfBookings = CALL readNumberOfBookings(bookingInputFile)
CALL readBookingInput(bookingInputFile, bookingList, numOfBookings)
CLOSE bookingInputFile
```

```
SET bookingOutputFile = OPEN passenger.out for appending
IF bookingOutputFile is NULL THEN
    DISPLAY "Error opening file"
    RETURN 1
END IF
```

```
CALL printBookingTableHeader(bookingOutputFile, bookingList, numOfBookings)
```

```
SET routeUpdateFile = OPEN route.txt for writing
IF routeUpdateFile is NULL THEN
    DISPLAY "Error opening file"
    RETURN 1
END IF
```

```
SET priceUpdateFile = OPEN price.txt for writing
IF priceUpdateFile is NULL THEN
```

```

    DISPLAY "Error opening file"
    RETURN 1 // Exit program with error code
END IF

SET summaryFile = OPEN summary.txt for writing
if summaryFile is NULL THEN
    DISPLAY "Error opening file"
    RETURN 1
END IF

SET origin to "Tabaco"
SET date to "05/09/2024"
SET discountRate to 18.45

INITIALIZE passengerSummary:
passengerSummary.ticketsNumTotal = 0
passengerSummary.regularTicketsTotal = 0
passengerSummary.discountTicketsTotal = 0
passengerSummary.totalSales = 0.0

FOR EACH route IN routeList DO
    FOR EACH price IN priceList DO
        SET passengerSummary.ticketCountRegular[route][price] to 0
        SET passengerSummary.ticketCountDiscount[route][price] to 0
    END FOR
END FOR

REPEAT
    DISPLAY main menu prompt:
    " 1. Passenger (Press 1)
    2. Operator (Press 2)
    3. Exit   (Press 3)
    -----
    Please enter your choice: "

    READ mainOption choice
    CASE mainOption OF:
        '1' :

            DISPLAY passenger dashboard prompt:
            READ bookingOption choice
            CASE bookingOption OF:
                '1':

                    IF numOfBookings >= MAX_PASSENGERS THEN
                        DISPLAY "Maximum number of passengers is reached."
                    ELSE

```

```
SET searchResult = 0
DISPLAY book ticket menu prompt
CALL displayRoute(routeList, numOfRoutes)
READ routeIdChoice
```

```
SET searchResult = CALL function: findDestinationByRouteId(routeList, numOfRoutes, routeIdChoice,
passengerInfo.destination)
```

```
IF searchResult is equal to 0 THEN
    DISPLAY "Route is not available at the moment."
    CONTINUE to next iteration
ELSE IF searchResult is equal to -1 THEN
    DISPLAY "Route ID not found."
    CONTINUE to next iteration
ELSE
```

```
REPEAT
    DISPLAY "Name (Put '-' between First name and Last Name): "
    READ passengerInfo.name
    DISPLAY "Select Bus Type:"
```

```
FOR i from 0 to numOfPrice - 1 DO
    Display bus type options
END FOR
```

```
DISPLAY "Bus Type: "
READ busTypeChoice
```

```
IF busTypeChoice is between 1 and numOfPrice THEN
    SET passengerInfo.busType = priceList[busTypeChoice - 1].busType
ELSE
    DISPLAY "Invalid choice. Try Again"
END IF
```

```
DISPLAY "Number of Passengers: "
READ passengerInfo.ticketsNum
DISPLAY " Number of Discounted Passengers: "
READ passengerInfo.discountTickets
```

```
SET passengerInfo.regTickets = passengerInfo.ticketsNum - passengerInfo.discountTickets
CALL calculatePayment (&passengerInfo, priceList, routeList, discountRate, routeIdChoice, numOfRoutes,
```

numOfPrice)

```
DISPLAY ticket summary
DISPLAY "Main Menu:
1. Confirm Booking
2. Cancel Booking
```

```
-----
Please enter your choice: "
READ printOption
```

```

CASE printOption OF:
    '1':
        DISPLAY "Booking Confirmed."
        CALL printTicket (&passengerInfo, ticketFile, routeIdChoice, origin, date)
        CALL addTicket (bookingInputFile, bookingOutputFile, numOfBookings, bookingList, &passengerInfo)
    '2':
        DISPLAY "Booking Cancelled."
    OTHERS:
        DISPLAY "Invalid choice. Try Again"
        CONTINUE to the next iteration

UNTIL printOption is equal to '2'
END IF
'2':
    CALL displayFAQs()
    CONTINUE to next iteration
'3':
    BREAK out of loop
OTHERS:
    DISPLAY "Invalid choice. Try Again"
UNTIL bookingOption is equal to '3'
BREAK out of loop

'2' :
REPEAT
    DISPLAY "Employee ID Number: "
    READ loggedOperator.idNumber
    DISPLAY " PIN: "
    READ loggedOperator.pin

    SET passResult = 0
    passResult = CALL searchOperator(&loggedOperator, operatorList, numOfOperator)

    IF passResult == 1 THEN
        REPEAT
            DISPLAY operator dashboard

            READ operatorOption

            CASE operatorOption OF:
                '1':
                    REPEAT
                        DISPLAY route menu
                        CALL displayOperatorRoute(routeList, numOfRoutes)
                        READ routeOption

                        CASE routeOption OF:

```

```

'1':
    INCREMENT numOfRoutes
    IF numOfRoutes >= MAX_ROUTE THEN
        DISPLAY "Maximum number of routes is reached."
    ELSE
        DISPLAY add route prompt
        CALL displayOperatorRoute(routeList, numOfRoutes)
        SET routeList[numOfRoutes - 1].routeRef = numOfRoutes
        DISPLAY "Route Destination: "
        READ routeList[numOfRoutes - 1].destination
        DISPLAY "Route Distance from Origin: "
        READ routeList[numOfRoutes - 1].distanceKm
        DISPLAY "Route Status(Available/Not-Available): "
        READ routeList[numOfRoutes - 1].status
        IF (routeList[numOfRoutes - 1].status is "Available" OR routeList[numOfRoutes - 1].status is
"Not-Available") THEN
            DISPLAY "Route is added."
        ELSE
            SET the status of the last route in routeList to "Not-Specified"
            DISPLAY "Invalid Status. Status must be either \"Available\" or \"Not-Available\"."
            CONTINUE to the next iteration
        ENDIF
    END IF
'2':
    DISPLAY list of routes
    DISPLAY "Select Route to edit:
        Route ID:"
    READ routeIdChoice
    DISPLAY current route destination
    Select part to edit:
    1. Distance from Origin
    2. Status
    3. Back to Menu
    -----
    Please enter your choice: "
    READ editOption

    CASE editOption OF:
        '1':
            DISPLAY "Current Distance from Origin: routeList[routeIdChoice - 1].distanceKm
                New Distance (km): "
            READ routeList[routeIdChoice - 1].distanceKm
            DISPLAY "Route Distance is changed."
        '2':
            DISPLAY "Current Status: routeList[routeIdChoice - 1].status
                New Status (Available/Not-Available): "
            READ routeList[routeIdChoice - 1].status
            DISPLAY "Route Status is changed."

```

```

        '3':
            BREAK out of loop
        OTHERS:
            DISPLAY "Invalid input. Try Again"
    '3':
        CALL displayOperatorRoute(routeList, numOfRoutes)
        DISPLAY "Select Route to delete:
            Route ID: "
        READ routeldChoice
        SET searchResult = CALL searchRoute(routeList, routeldChoice, numOfRoutes)
        IF searchResult is equal to 1 THEN
            CALL swapRoute(&routeList[routeldChoice - 1], &routeList[numOfRoutes - 1])
            DECREMENT numOfRoutes
            DISPLAY "Route is deleted."
        ELSE
            DISPLAY "Route ID not found"
        END IF
    '4':
        DISPLAY current origin
        READ new origin
        DISPLAY "Starting point is changed."
    '5':
        DISPLAY current date
        READ new date
        DISPLAY "Date is changed."
    '6':
        BREAK out of loop
    OTHERS:
        DISPLAY "Invalid choice. Try again"
UNTIL routeOption is equal to '6'

'2':
REPEAT
    CALL displayPrice(priceList, numOfPrice)
    READ priceOption

CASE priceOption OF:
    '1':
        DISPLAY current discountRate
        READ new discountRate
        DISPLAY "Discount Rate is changed."
    '2':
        INCREMENT numOfPrice
        IF numOfPrice >= MAX_CATEGORY THEN
            DISPLAY "Maximum number of category is reached"
        ELSE
            DISPLAY categories with rates
            SET priceList[numOfPrice - 1].priceRef = numOfPrice

```



```

        DISPLAY "Price Category: "
        READ priceList[numOfPrice - 1].busType
        DISPLAY "Price Rate (Php/km): "
        READ priceList[numOfPrice - 1].pricePhpKm
        DISPLAY "-----
            Price Category is added."
    END IF
'3':
    CALL displayPrice(priceList, numOfPrice)
    DISPLAY "Select Category to edit:
        Price ID: "
    READ priceIdChoice
    DISPLAY "=====
        Category: priceList[priceIdChoice - 1].busType
        Current Price Rate (Php/km): priceList[priceIdChoice - 1].pricePhpKm
        New Price Rate (Php/km): "
    READ priceList[priceIdChoice - 1].pricePhpKm
    DISPLAY "-----
        Price Category is changed."
'4':
    CALL displayPrice(priceList, numOfPrice)
    DISPLAY "Select Category to delete:
        Price ID: "
    READ priceIdChoice
    SET searchResult = 0
    searchResult = CALL searchCategory (priceList, priceIdChoice, numOfPrice)
    IF searchResult is equal to 1 THEN
        CALL swapCategory (&priceList[priceIdChoice - 1], &priceList[numOfPrice - 1])
        DECREMENT numOfPrice
        DISPLAY "-----
            Category is deleted."
    ELSE
        DISPLAY "Route ID not found."
    END IF
'5':
    BREAK out of loop
OTHERS:
    DISPLAY "Invalid choice. Try again"
UNTIL priceOption is equal to '5'
'3':
    FOR i = 0 TO numOfBookings - 1 DO
        DISPLAY list of passengers/tickets:
        CALL summarizePassengerList(bookingList[i], passengerSummary)
        CALL countTicketsPerRoute(bookingList[i], passengerSummary, routeList, priceList, numOfRoutes,
numOfPrice)
    END FOR

    DISPLAY "Print Summary? (1 for YES, 0 for NO): "

```

```

        READ printSummaryChoice
        IF printSummaryChoice is equal to '1' THEN
            CALL printSummary (summaryFile, &passengerSummary, numOfRoutes, numOfPrice, date,
numOfBookings, routeList, priceList)
            CLOSE summaryFile
            DISPLAY "Summary is printed (Please check summary.txt)\n"
        ELSE IF printSummaryChoice is not equal to '0' THEN
            DISPLAY "Invalid input."
            CONTINUE to next iteration
        END IF
    '4':
        DISPLAY account information: Name, employee ID, pin
        DISPLAY operator instructions: CALL displayInstruction()
    '5':
        CALL updateRoute(routeUpdateFile, numOfRoutes, routeList)
        CLOSE routeUpdateFile
        CALL updatePrice(priceUpdateFile, numOfPrice, priceList)
        CLOSE routeUpdateFile
        BREAK out of loop
    OTHERS:
        DISPLAY "Invalid choice. Try again"
    UNTIL operatorOption == '5'
ELSE IF passResult == 2 THEN
    DISPLAY "Incorrect PIN. Try Again"
    BREAK out of loop
ELSE
    DISPLAY "Employee ID not found. Try Again"
    BREAK out of loop
END IF
UNTIL operatorOption is equal to '5'

'3' :
    BREAK out of loop
OTHERS:
    DISPLAY "Invalid choice. Try again"
UNTIL mainOption is equal to '3'

```

// Functions used

```

FUNCTION printTicket(passengerInfo: Passenger, ticketFile: FILE, routeId: int, origin: string, date: string)
    DISPLAY ticket: // DISPLAY a format of a ticket
END FUNCTION

```

```

FUNCTION searchOperator(operatorCheck: Operator, operatorList: Operator[], numOperator: int) -> int
    FOR each operator IN operatorList DO
        IF operator.idNumber EQUALS operatorCheck.idNumber AND operator.pin EQUALS operatorCheck.pin THEN
            RETURN 1
        END IF
    END IF

```

```
END FOR
RETURN 0
END FUNCTION
```

```
FUNCTION searchRoute(routeList: Route[], routeIdCheck: int, numOfRoutes: int) -> int
FOR each route IN routeList DO
    IF route.routeRef EQUALS routeIdCheck THEN
        RETURN 1
    END IF
END FOR
RETURN 0
END FUNCTION
```

```
FUNCTION searchCategory(priceList: Price[], priceIdCheck: int, numOfPrice: int) -> int
FOR each category IN priceList DO
    IF category.priceRef EQUALS priceIdCheck THEN
        RETURN 1
    END IF
END FOR
RETURN 0
END FUNCTION
```

```
FUNCTION swapRoute(route1: Route, route2: Route)
SET TEMP_ROUTE = route1
SET route1 = route2
SET route2 = TEMP_ROUTE
END FUNCTION
```

```
FUNCTION summarizePassengerList(passenger: Passenger, summaryPassenger: Summary)
SET summaryPassenger.ticketsNumTotal += passenger.ticketsNum
SET summaryPassenger.regularTicketsTotal += passenger.regTickets
SET summaryPassenger.discountTicketsTotal += passenger.discountTickets
SET summaryPassenger.totalSales += passenger.totalPayment
END FUNCTION
```

```
FUNCTION countTicketsPerRoute(passenger: Passenger, summaryPassenger: Summary, routeList: Route[], priceList: Price[],
numOfRoutes: int, numOfPrice: int)
FOR each route IN routeList DO
    IF passenger.destination EQUALS route.destination THEN
        FOR each category IN priceList DO
            IF passenger.busType EQUALS category.busType THEN
                SET summaryPassenger.ticketCountRegular[routeList.index][priceList.index] += passenger.regTickets
                SET summaryPassenger.ticketCountDiscount[routeList.index][priceList.index] += passenger.discountTickets
                BREAK out of loop
            END IF
        END FOR
    END IF
END FOR
END IF
END FOR
```

END FUNCTION

FUNCTION printSummary(summaryFile: FILE, summaryPassenger: Summary, numOfRoutes: int, numOfPrice: int, date: string, numOfPassenger: int, routeList: Route[], priceList: Price[])

 DISPLAY summary:

END FUNCTION

FUNCTION calculatePayment(passengerInfo: Passenger, priceList: Price[], routeList: Route[], discountRate: float, routeIdChoice: int, numOfRoutes: int, numOfPrice: int)

 FOR each route IN routeList DO

 IF route.routeRef EQUALS routeIdChoice THEN

 FOR each category IN priceList DO

 IF passengerInfo.busType EQUALS category.busType THEN

 SET passengerInfo.ticketPrice = route.distanceKm * category.pricePhpKm

 BREAK out of loop

 END IF

 END FOR

 BREAK out of loop

 END IF

END FOR

 passengerInfo.totalPayment = (passengerInfo.ticketPrice * passengerInfo.regTickets) + ((passengerInfo.ticketPrice - (passengerInfo.ticketPrice * (discountRate / 100.00))) * passengerInfo.discountTickets)

END FUNCTION

FUNCTION findDestinationByRouteId(routeList: Route[], numOfRoutes: int, routeIdChoice: int, passengerDestination: string)
-> integer:

 FOR i FROM 0 TO numOfRoutes - 1 DO

 IF routeList[i].routeRef EQUALS routeIdChoice THEN

 IF routeList[i].status EQUALS "Not-Available" OR routeList[i].status EQUALS "Not-Specified" THEN

 RETURN 0

 ELSE

 SET passengerDestination TO routeList[i].destination

 RETURN 1

 END IF

 END IF

 END FOR

 RETURN -1

END FUNCTION

FUNCTION readOperatorFile(operatorFile: FILE, operatorList: Operator[], numOfOperator: int)

 FOR i FROM 0 TO numOfOperator DO

 READ operatorList[i].idNumber, operatorList[i].firstName, operatorList[i].lastName, operatorList[i].pin FROM operatorFile

 END FOR

 CLOSE operatorFile

END FUNCTION

FUNCTION readRouteFile(routeFile: FILE, routeList: Route[], numOfRoutes: int)

 FOR i FROM 0 TO numOfRoutes DO

```
    READ routeList[i].routeRef, routeList[i].destination, routeList[i].distanceKm, routeList[i].status FROM routeFile
END FOR
CLOSE routeFile
END FUNCTION
```

```
FUNCTION readPriceFile(pricingFile: FILE, priceList: Price[], numOfPrice: int)
    FOR i FROM 0 TO numOfPrice DO
        READ priceList[i].priceRef, priceList[i].busType, priceList[i].pricePhpKm FROM pricingFile
    END FOR
    CLOSE pricingFile
END FUNCTION
```

```
FUNCTION readBookingInput(bookingInputFile: FILE, bookingList: Passenger[], numOfBookings: int)
    IF numOfBookings > 0 THEN
        FOR i FROM 0 TO numOfBookings DO
            READ bookingList[i].passengerID, bookingList[i].name, bookingList[i].destination, bookingList[i].busType,
            bookingList[i].ticketPrice, bookingList[i].ticketsNum, bookingList[i].regTickets, bookingList[i].discountTickets,
            bookingList[i].totalPayment FROM bookingInputFile
        END FOR
    END IF
END FUNCTION
```

```
FUNCTION addTicket(bookingInputUpdateFile: FILE, bookingOutputFile: FILE, numOfBookings: int, bookingList: Passenger[],
passengerInfo: Passenger)
    numOfBookings++
    bookingList[numOfBookings - 1] = passengerInfo
    DISPLAY passengerInfo TO bookingInputUpdateFile
    DISPLAY passengerInfo TO bookingOutputFile
END FUNCTION
```

```
FUNCTION updatePrice(priceFile: FILE, numOfPrice: int, priceList: Price[])
    DISPLAY numOfPrice TO priceFile
    FOR i FROM 0 TO numOfPrice DO
        DISPLAY priceList[i] TO priceFile
    END FOR
END FUNCTION
```

```
FUNCTION updateRoute(routeFile: FILE, numOfRoute: int, routeList: Route[])
    DISPLAY numOfRoute TO routeFile
    FOR i FROM 0 TO numOfRoute DO
        DISPLAY routeList[i] TO routeFile
    END FOR
END FUNCTION
```

```
FUNCTION displayInstruction()
    // DISPLAY Texts of instructions, faqs etc.
END FUNCTION
```

```

FUNCTION displayFAQs()
    // DISPLAY Texts of instructions, faqs etc.
END FUNCTION

FUNCTION displayRoute(routeList: Route[], numOfRoutes: int)
    FOR i FROM 0 TO numOfRoutes - 1 DO
        DISPLAY routeList[i].routeID, routeList[i].destination, routeList[i].status
    END FOR
END FUNCTION

FUNCTION displayOperatorRoute(routeList: Route[], numOfRoutes: int)
    FOR i FROM 0 TO numOfRoutes - 1 DO
        DISPLAY routeList[i].routeID, routeList[i].destination, routeList[i].distance, routeList[i].status
    END FOR
END FUNCTION

FUNCTION displayPrice(priceList: Price[], numOfPrice: int)
    FOR i FROM 0 TO numOfPrice - 1 DO
        DISPLAY priceList[i].priceID, priceList[i].category, priceList[i].rate
    END FOR
END FUNCTION

```

III. Program Specification

A. Standard C Functions Used

Here are the predefined functions we've used in the program:

Header File	Predefined functions used
<i>stdio.h</i>	<i>printf</i> <i>fprintf</i> <i>scanf</i> <i>fscanf</i> <i>fopen</i> <i>fclose</i>
<i>string.h</i>	<i>strcmp</i> <i>strcpy</i>

B. Functions

The following are the detailed description of each user-defined function in our program. Each description provides the purpose, parameters used, return value of the function, local variables used, and the main operations inside the function.

1. printTicket

- Purpose: Prints a ticket for a passenger with destination, number of tickets, and payment price details.
- Parameters:
 - Passenger *passengerInfo: Pointer to a structure containing passenger information.

- FILE *ticketFile: Pointer to a file where the ticket will be printed.
- int routeId: ID of the route.
- char *origin: Origin of the route.
- char *date: Date of the ticket.
- Return Value: None (void).
- Local Variables: None.
- Operations: Prints ticket details including passenger name, date, route details, ticket price, ticket summary, and total price to the provided file stream (ticketFile).

2. **searchOperator**

- Purpose: Searches for an operator by employee ID and PIN.
- Parameters:
 - Operator *operatorCheck: Pointer to a structure containing the operator to be checked.
 - Operator *operatorList: Pointer to an array of structures containing the list of operators.
 - int numOfOperator: Number of operators in the list.
- Return Value: An integer indicating the search result:
 - 0: No match found.
 - 1: Operator found.
 - 2: ID matched but PIN didn't.
- Local Variables: integer flag to store the search result.
- Operations: Iterates through the list of operators, compares ID and PIN, and updates the flag accordingly.

3. **searchRoute**

- Purpose: Searches for a route by ID.
- Parameters:
 - Route *routeList: Pointer to an array of structures containing the list of routes.
 - int routeIdCheck: ID of the route to be checked.
 - int numOfRoutes: Number of routes in the list.
- Return Value: An integer indicating the search result:
 - 0: No match found.
 - 1: Route found.
- Local Variables: integer flag to store the search result.
- Operations: Iterates through the list of routes, compares route IDs, and updates the flag accordingly.

4. **searchCategory**

- Purpose: Searches for a category by ID.
- Parameters:
 - Price *priceList: Pointer to an array of structures containing the list of price categories.
 - int priceIdCheck: ID of the category to be checked.
 - int numOfPrice: Number of price categories in the list.
- Return Value: An integer indicating the search result:
 - 0: No match found.
 - 1: Category found.
- Local Variables: int flag to store the search result.

- Operations: Iterates through the list of price categories, compares category IDs, and updates the flag accordingly.

5. **swapRoute**

- Purpose: Swaps two route structures.
- Parameters:
 - Route *route1: Pointer to the first route structure.
 - Route *route2: Pointer to the second route structure.
- Return Value: None (void).
- Local Variables: Route tempRoute to temporarily store route information during swapping.
- Operations: Swaps destination, distance, and status between the two route structures.

6. **summarizePassengerList**

- Purpose: Updates a summary structure with passenger data.
- Parameters:
 - Passenger *passenger: Pointer to a structure containing passenger information.
 - Summary *summaryPassenger: Pointer to a structure where passenger data will be summarized.
- Return Value: None (void).
- Local Variables: None.
- Operations: Updates the summary structure (summaryPassenger) with the ticket count, regular tickets, discounted tickets, and total sales from the provided passenger structure (passenger).

7. **countTicketsPerRoute**

- Purpose: Counts tickets per route and category and updates the summary structure.
- Parameters:
 - Passenger *passenger: Pointer to a structure containing passenger information.
 - Summary *summaryPassenger: Pointer to a structure where ticket counts will be updated.
 - Route *routeList: Pointer to an array of structures containing the list of routes.
 - Price *priceList: Pointer to an array of structures containing the list of price categories.
 - int numOfRoutes: Number of routes in the list.
 - int numOfPrice: Number of price categories in the list.
- Return Value: None (void).
- Local Variables: None.
- Operations: Iterates over the routes and price categories, compares passenger destination and bus type with route and category details, and updates the ticket counts in the summary structure accordingly.

8. **printSummary**

- Purpose: Prints a summary of ticket sales to a file.
- Parameters:
 - FILE *summaryFile: Pointer to a file where the summary will be printed.
 - Summary *summaryPassenger: Pointer to a structure containing the summary of ticket sales.

- int numOfRoutes: Number of routes in the list.
- int numOfPrice: Number of price categories in the list.
- char *date: Date for which the summary is generated.
- int numOfPassenger: Total number of passengers.
- Route *routeList: Pointer to an array of structures containing the list of routes.
- Price *priceList: Pointer to an array of structures containing the list of price categories.
- Return Value: None (void).
- Local Variables: None.
- Operations: Prints a detailed summary of ticket sales including total number of passengers, total tickets sold, total sales amount, and ticket counts per route and category to the provided file stream (summaryFile).

9. calculatePayment

- Purpose: Calculates payment for a passenger based on route and category.
- Parameters:
 - Passenger *passengerInfo: Pointer to a structure containing passenger information.
 - Price *priceList: Pointer to an array of structures containing the list of price categories.
 - Route *routeList: Pointer to an array of structures containing the list of routes.
 - float discountRate: Discount rate applicable to the passenger.
 - int routeIdChoice: ID of the chosen route.
 - int numOfRoutes: Number of routes in the list.
 - int numOfPrice: Number of price categories in the list.
- Return Value: None (void).
- Local Variables: int routeFlag, int priceFlag to indicate whether route and category were found.
- Operations: Calculates ticket price based on route and category, applies discounts if applicable, and calculates the total payment for the passenger.

10. findDestinationByRouteId

- Purpose: Finds the destination by route ID and checks its availability.
- Parameters:
 - Route *routeList: Pointer to an array of structures containing the list of routes.
 - int numOfRoutes: Number of routes in the list.
 - int routeIdChoice: ID of the chosen route.
 - char *passengerDestination: Pointer to a character array where the destination will be stored.
- Return Value: An integer indicating the search result:
 - 0: Route not available.
 - 1: Destination is found and set.
 - -1: Route ID not found.
- Local Variables: None.
- Operations: Locates the route corresponding to the given ID, confirms its availability, and updates the passenger destination if the route exists or is unavailable.

11. displayFAQs

- Purpose: Displays frequently asked questions and passenger instructions.
- Parameters: None.
- Return Value: None (void).
- Local Variables: None.
- Operations: Prints a series of FAQs regarding booking, routes, cancellation, payment, and contact information, followed by passenger instructions on using the system.

12. readOperatorFile

- Purpose: Reads operator data from a file.
- Parameters:
 - FILE *operatorFile: Pointer to the file containing operator data.
 - Operator *operatorList: Pointer to an array of structures to store operator data.
 - int numOfOperator: Number of operators in the file.
- Return Value: None (void).
- Local Variables: None.
- Operations: Reads operator data from the provided file stream (operatorFile) and stores it in the array of operator structures (operatorList).

13. readRouteFile

- Purpose: Reads route data from a file.
- Parameters:
 - FILE *routeFile: Pointer to the file containing route data.
 - Route *routeList: Pointer to an array of structures to store route data.
 - int numOfRoutes: Number of routes in the file.
- Return Value: None (void).
- Local Variables: None.
- Operations: Reads route data from the provided file stream (routeFile) and stores it in the array of route structures (routeList).

14. readPriceFile

- Purpose: Reads price data from a file.
- Parameters:
 - FILE *pricingFile: Pointer to the file containing price data.
 - Price *priceList: Pointer to an array of structures to store price data.
 - int numOfPrice: Number of price categories in the file.
- Return Value: None (void).
- Local Variables: None.
- Operations: Reads price data from the provided file stream (pricingFile) and stores it in the array of price structures (priceList).

15. readBookingInput

- Purpose: Reads booking input from a file.
- Parameters:
 - FILE *bookingInputFile: Pointer to the file containing booking input data.
 - Passenger *bookingList: Pointer to an array of structures to store booking data.
 - long int numOfBookings: Number of bookings in the file.
- Return Value: None (void).
- Local Variables: None.
- Operations: Reads booking input data from the provided file stream (bookingInputFile) and stores it in the array of passenger structures (bookingList).

16. addTicket

- Purpose: Adds a ticket to the booking list and updates input and output files.
- Parameters:
 - FILE *bookingInputUpdateFile: Pointer to the file for updating booking input data.
 - FILE *bookingOutputFile: Pointer to the file for printing booking output data.
 - int numOfBookings: Number of bookings in the list.
 - Passenger *bookingList: Pointer to an array of structures containing booking data.
 - Passenger *passengerInfo: Pointer to a structure containing passenger information to be added as a ticket.
- Return Value: None (void).
- Local Variables: None.
- Operations: Appends the new ticket (passenger information) to the booking list, updates the booking input file with the new ticket data, and prints the ticket details to the booking output file.

17. displayRoute

- Purpose: Displays route details.
- Parameters:
 - Route *routeList: Pointer to an array of structures containing route data.
 - int numOfRoutes: Number of routes in the list.
- Return Value: None (void).
- Local Variables: None.
- Operations: Prints the route ID, destination, and status of each route stored in the array of route structures (routeList).

18. displayOperatorRoute

- Purpose: Displays detailed route information including distance.
- Parameters:
 - Route *routeList: Pointer to an array of structures containing route data.
 - int numOfRoutes: Number of routes in the list.
- Return Value: None (void).
- Local Variables: None.
- Operations: Prints the route ID, destination, distance, and status of each route stored in the array of route structures (routeList).

19. displayPrice

- Purpose: Displays price details.
- Parameters:
 - Price *priceList: Pointer to an array of structures containing price data.
 - int numOfPrice: Number of price categories in the list.
- Return Value: None (void).
- Local Variables: None.
- Operations: Prints the price ID, category, and rate of each price category stored in the array of price structures (priceList).

20. displayInstruction

- Purpose: Displays operator instructions.
- Parameters: None.
- Return Value: None (void).
- Local Variables: None.

- Operations: Prints instructions for operators on accessing the dashboard, managing routes, prices and discounts, viewing passenger lists, accessing account information, and logging out.

21. updatePrice

- Purpose: Updates price details in the price file.
- Parameters:
 - FILE *priceFile: Pointer to the file for updating price data.
 - int numOfPrice: Number of price categories.
 - Price *priceList: Pointer to an array of structures containing price data.
- Return Value: None (void).
- Local Variables: None.
- Operations: Updates the price data in the provided file stream (priceFile) with the current price list stored in the array of price structures (priceList).

22. updateRoute

- Purpose: Updates route details in the route file.
- Parameters:
 - FILE *routeFile: Pointer to the file for updating route data.
 - int numOfRoute: Number of routes.
 - Route *routeList: Pointer to an array of structures containing route data.
- Return Value: None (void).
- Local Variables: None.
- Operations: Updates the route data in the provided file stream (routeFile) with the current route list stored in the array of route structures (routeList).

IV. User Manual

1. Compilation:

- Open a terminal or command prompt window.
- Navigate to the directory where the C source code file is located.
- Compile the source code using a C compiler such as GCC (GNU Compiler Collection) with the following command:

```
gcc -o bus_system bus_system.c
```

- This command will compile the source code file `bus\_system.c` and create an executable file named `bus\_system`.

2. Execution:

- **NOTE:** Before running the program, ensure the following files are available in the same directory:
 - `operators.txt`: Contains operator data.
 - `route.txt`: Contains route data.
 - `price.txt`: Contains price category data.
 - `passenger.in`: Contains booking input data.
 - `tickets.txt`: Will be created by the program to store tickets to be printed.
 - `passenger.out`: Will be created by the program to store booking output data.
 - `summary.txt`: Will be created by the program to store summary of tickets.

These files are necessary for the program to function properly. Make sure they are present and accessible before executing the program.

- Additionally, if you wish to provide custom data:
 - Operators Data (operators.txt):
 - Open the operators.txt file using a text editor.
 - Enter operator details in the following format for each operator:

ID	First Name	Last Name	PIN
----	------------	-----------	-----
 - Replace ID , First Name , Last Name , and PIN with the respective details of each operator. One line per each data group.
 - Save the file after entering all the operator details.
 - Route Data (route.txt):
 - Open the route.txt file using a text editor.
 - Enter route details in the following format for each route:

Route ID	Destination	Distance(Km)	Status
----------	-------------	--------------	--------
 - Replace Route ID , Destination , Distance , and Status with the respective details of each route. One line per each data group.
 - Save the file after entering all the route details.
 - Price Data (price.txt):
 - Open the price.txt file using a text editor.
 - Enter price details in the following format for each price category:

Price ID	Category	Price(Php/Km)
----------	----------	---------------
 - Replace Price ID , Category , and Price with the respective details of each price category. One line per each data group.
 - Save the file after entering all the price details.
- After successful compilation and file preparation, execute the program by running the generated executable file:
`./bus_system.exe`
- This command will start the program and display the main menu on the terminal.

3. Main Menu:

- Upon execution, the program will display a main menu with several options.
 - **Passenger:** Press '1' to access the Passenger Dashboard for booking tickets.
 - **Operator:** Press '2' to log in as an operator for route management and other administrative tasks.
 - **Exit:** Press '3' to exit the Ticket Booking System.
- The user can interact with the program by entering the corresponding option number followed by pressing the Enter key.

4. Passenger Side:

- **Passenger Dashboard**

After selecting the 'Passenger' option, users are directed to the Passenger Dashboard. Here, users can:

1. View available options: Book Ticket, FAQs, and Exit.
2. Select the desired action by entering the corresponding number.

- **Booking Ticket**

Upon selecting the 'Book Ticket' option, users can proceed to book tickets for their journey.

1. Follow the prompts to:
2. Choose the destination route.
3. Select the bus type.
4. Specify the number of passengers, including regular and discounted tickets.
5. Confirm or cancel the booking.

- **Printable Tickets**

After confirming the booking, users receive printable tickets containing essential journey details. These tickets can be easily printed for convenient access.

- **Exiting the System**

Users can exit the Ticket Booking System by selecting the 'Exit' option from the main menu.

5. Operator Side:

- **Accessing the Operator Dashboard**

Log in to the Operator Dashboard using your employee ID and PIN.

- **Operator Dashboard Options**

1. **Routes:** Manage routes, including adding, editing, or deleting routes, and changing the starting point and date.
2. **Prices and Discounts:** Manage price categories and discount rates, including adding, editing, or deleting categories and changing the discount rate.
3. **Passenger List:** View a list of passengers, including ticket details and sales summaries.
4. **Account Information:** Access account details and program instructions.
5. **Logout:** Securely log out from the Operator Dashboard.

- **Routes Management**

1. **Add Route:** Add a new route, specifying the destination, distance, and status.
2. **Edit Route:** Modify existing route details, such as distance and status.
3. **Delete Route:** Remove routes from the system, ensuring proper handling of route data.

- **Prices and Discounts Management**

1. **Change Discount Rate:** Modify the discount rate applied to ticket prices.
2. **Add Category:** Add new price categories for different bus types.
3. **Edit Category:** Modify existing price categories, adjusting price rates as needed.
4. **Delete Category:** Remove price categories that are no longer required.

- **Passenger List**

1. View a comprehensive list of passengers, including ticket details such as destination, bus type, and ticket price.
2. Access sales summaries and ticket count per route and category for better insight into passenger demographics and sales trends.

- **Account Information**

1. Access account details, including operator ID, name, and PIN.

2. View program instructions and guidelines for operating the Ticket Booking System effectively.
- **Logout**
Securely log out from the Operator Dashboard to protect operator accounts and system data.

V. Member Participation

Members	Roles and Contributions
Melbert Bonina	Made the C program and the pseudo code of the following parts: ● Passenger Dashboard Functionalities: ○ Book Ticket ● Operator Dashboard Functionalities: ○ Manage Routes ○ Manage Prices and Discounts ● File Handling Implementations: ○ Read Data ○ Write Data ○ Update Data ● Declaration of Variables, arrays, and structs used ● Operator Side Functions: 1. searchOperator 2. displayOperatorRoute 3. swapRoute 4. swapCategory 5. updateRoute 6. updatePrice ● Passenger Side Functions: ○ findDestinationByRouteId ○ calculatePayment ○ printTicket ○ addTicket
Maria Francia Abdula	Made the C program and the pseudo code of the following parts: ● Manage Discounts ● Passenger Output File Handling ● Update Passenger Remarks ● FAQs and Instructions ● Contact Support ● General Functions ○ countTicketsPerRoute ○ printSummary ○ displayInstruction
Angelica Bilaos	Made the C program and the pseudo code of the following parts:

	● Passenger Menu ● Passenger List Summary ● Passenger Summary Input Update File Handling ● User Manual ● General Functions ○ summarizePassengerList ○ displayRoute ○ displayPrice
Rhoda Rodriguez	Made the C program and the pseudo code of the following parts: ● Operator Log-in ● Account Info Menu ● Operator Instructions ● FAQs and Instructions ● General Functions ○ searchRoute ○ searchCategory ○ displayFAQs